

Full Stack Development with MERN

Flight Booking and Management System

1. Introduction

1.1 Project Title

Flight Booking and Management System

1.2 Member

Name: Karthikeyan S

Role: Full Stack Developer

Requirements:

- Developed the full React frontend for customer, admin, and flight-operator modules.
- Designed landing pages, flight search, booking forms, and dashboards.
- Connected the frontend with backend REST APIs using Axios.
- Implemented JWT-based login, authentication, and role-based access.
- Designed MongoDB schemas for users, flights, bookings, and operators.
- Built backend APIs using Node.js and Express for all major operations.
- Added features like seat selection, fare calculation, passenger handling, and cancellations.
- Created admin dashboards to display user stats, bookings, flights, and operator requests.
- Developed operator features to add/edit flights and manage their bookings.

2. Project Overview

2.1 Purpose of the Project

Flight booking system is a MERN-based online platform designed to simplify the process of booking flights, managing reservations, and administering flight operations. The system allows customers to search for flights, book tickets, and view booking history. It also provides separate dashboards for Admins and Flight Operators to manage flights, users, bookings, and operational approvals.

The purpose is to deliver a **scalable, secure, and user-friendly flight reservation system**.

2.2 Key Features

Customer Features

- User registration and login
- Search flights (one-way & return)
- View available flights based on date & destination
- Book flights by adding passenger details
- View booking history
- Cancel bookings

Admin Features

- View total users, operators, flights, and booking statistics
- Approve or reject flight operator registrations
- Manage all users and bookings
- Monitor flight details

Flight Operator Features

- Request approval from admin
- Add new flights
- Edit flight information
- View bookings for their own flights
- Dashboard showing flight statistics

System Features

- JWT-based secure authentication
- Role-based authorization (Admin, Customer, Flight Operator)
- Real-time validation
- REST API backend
- Responsive UI

3. Architecture

3.1 Frontend Architecture (React.js)

The frontend is built using **React.js** with the following structure:

- **Pages:** Landing, Login, Register, Book Flight, Admin Dashboard, Operator Dashboard
- **Components:** Navbar, Forms, Cards, Listings
- **Context API:** Centralized global state for user authentication, inputs, and token handling
- **Routing:**
 - `react-router-dom` for client-side routing
 - Protected routes (AuthProtector, LoginProtector)
- **CSS:** Custom styles + Bootstrap for components

The frontend consumes backend APIs using **Axios**.

3.2 Backend Architecture (Node.js + Express.js)

The backend is built on **Express.js**, exposing RESTful endpoints for:

- Authentication (login/register)
- User management
- Flight CRUD operations
- Booking management
- Operator approval system

The backend includes:

- **Middleware** for authentication (JWT)
- **Role-based route protection**
- **Controllers** for specific modules
- **Services** for reusable logic

3.3 Database Architecture (MongoDB)

MongoDB stores:

User Schema

- Username
- Email
- Password (hashed)
- User type (customer, admin, operator)
- Approval status (for operators)

Flight Schema

- Flight name & ID
- Origin & destination
- Departure & arrival time
- Base fare
- Total seats

Booking Schema

- User ID
- Flight ID
- Passenger list
- Journey date & time
- Total price
- Status (confirmed/cancelled)

Each collection is indexed for fast search queries.

4. Setup Instructions

4.1 Prerequisites

Install the following:

- **Node.js** (v16+)
- **MongoDB** (local or cloud – Atlas)
- **npm / yarn**
- Code editor (VS Code recommended)

4.2 Installation Steps

Step 1 – Clone Repository

```
git clone https://github.com/your-repo-url.git
```

Step 2 – Install Frontend

```
cd client  
npm install
```

Step 3 – Install Backend

```
cd server  
npm install
```

Step 4 – Setup Environment Variables

Create a `.env` file in the server:

```
PORT=6001  
MONGO_URL=mongodb://localhost:27017/flightapp  
JWT_SECRET=your_secret_key
```

5. Folder Structure

5.1 Client Folder (React Frontend)

```
client/  
├── src/  
│   ├── components/  
│   ├── pages/  
│   ├── context/  
│   ├── RouteProtectors/  
│   ├── styles/  
│   ├── App.js  
│   └── index.js  
└── public/
```

Key Points:

- `components/` contains UI reusable components
- `pages/` contains all major screens
- `context/GeneralContext.jsx` handles auth, tokens, and state
- `RouteProtectors/` ensures restricted routing

5.2 Server Folder (Backend)

```
server/  
|— controllers/  
|— models/  
|— routes/  
|— middleware/  
|— server.js  
|— .env
```

Key Structure:

- Controllers contain business logic
- Routes map URLs to controllers
- Models define database schemas
- Middleware handles auth (JWT)

6. Running the Application

Start Backend

```
cd server  
npm start
```

Start Frontend

```
cd client
npm start
```

The application runs at:

- ➔ **Frontend:** <http://localhost:3000>
- ➔ **Backend:** <http://localhost:6001>

7. API Documentation

Below is an overview (you can expand further in your Word file):

Auth APIs

Method	Endpoint	Description
POST	<code>/register</code>	Create new user
POST	<code>/login</code>	Login user + return JWT token

Flight APIs

Method	Endpoint	Description
GET	<code>/fetch-flights</code>	Get all flights
GET	<code>/fetch-flight/:id</code>	Get specific flight
POST	<code>/add-flight</code>	Add new flight
PUT	<code>/update-flight</code>	Update existing flight

Booking APIs

Method	Endpoint	Description
POST	<code>/book-ticket</code>	Book flight
GET	<code>/fetch-bookings</code>	Get all bookings
PUT	<code>/cancel-ticket/:id</code>	Cancel a booking

Admin APIs

Method	Endpoint	Description
GET	<code>/fetch-users</code>	Get all users
POST	<code>/approve-operator</code>	Approve operator
POST	<code>/reject-operator</code>	Reject operator

8. Authentication Overview

Authentication is handled using:

- **JWT Tokens** for verifying logged-in users
- **LocalStorage** for token persistence
- **Express Middleware** to protect backend routes
- **Role-based routing** for:
 - customer
 - Admin
 - flight operator

Security Measures:

- Password hashing using bcrypt
- Authorization headers in Axios interceptor
- Auto-logout when token is missing

9. User Interface Overview

Landing Page

The landing page is the first screen the user sees.

It contains:

- A hero banner section introducing the platform.
- A flight search panel where users can enter “From”, “To”, date, and class.
- Available flights shown in card format after searching.

Login & Register Pages

These pages allow users, admins, and operators to log in.

They include:

- Email and password fields
- Validation messages
- Links to switch between register and login pages

Customer Dashboard

After login, customers can:

- Search for flights using filters
- View available flights in a list
- Book tickets through a simple form
- See their previous and upcoming bookings in one place

Admin Dashboard

The admin dashboard provides a quick overview of the system:

- Total registered users
- Total bookings
- Number of available flights
- New operator approval requests

Admins can also navigate to pages for managing users, bookings, and flights.

Flight Operator Dashboard

Operators have a separate interface where they can:

- Add new flights
- Edit existing flight details
- View their submitted flights
- Check approval status from the admin

10. Testing

Testing includes:

Manual Testing

- Form validations
- Searching flights
- Booking & cancellation
- Operator approval workflows
- Protected route testing

Automated Testing (Optional)

- Postman API testing
- Unit testing for controllers (Jest, Mocha – optional)

11. Demo

- A link to a **demo video** hosted on Google Drive..

Link:

[Flight booking.mp4](#)

12. Known Issues

- Flight seat availability is not updated in real time
- No pagination in listing pages
- Limited validation messages in some forms
- No email notification system implemented

13. Future Enhancements

- Real-time flight availability
- Seat selection system
- Payment gateway integration
- Email/SMS ticket confirmation
- Chatbot customer support
- Analytics dashboard for admin
- Multi-language support
- Mobile app using React Native